

Feature Selection in Statistical Machine Learning

Garvesh Raskutti

Lecture 16

Recap and outline

- Undirected and directed graphical models
- Directed acyclic graphical (DAG) models
- Examples: Time series and structural equation models (causal)
- DAG models and feature selection
- Comparing UGs and DAGs
- Examples

Undirected graphical models

Undirected graphical models:

$$G = (V, E)$$

where $V = (X_1, X_2, \dots, X_p)$, E undirected edges

Conditional independence relationship:

$$(X_j, X_k) \in E \Leftrightarrow X_j \not\perp\!\!\!\perp X_k | \text{All other features}$$

In words: There exists an edge $(X_j, X_k) \in E$ if and only if you condition on all other nodes/random variables, X_j and X_k are dependent

Undirected graphical models and feature selection

A neighborhood set for X_k is:

$$\mathcal{N}(X_k) = \{\ell : \ell \in \{1, 2, \dots, p\} \setminus \{k\}, X_k \not\perp\!\!\!\perp X_\ell | X_S \text{ where } S = \{1, 2, \dots, p\} \setminus \{k, \ell\}\}$$

For feature selection of Y recall that:

$$\tilde{S} = \{j : j \in \{1, 2, \dots, p\}, Y \not\perp\!\!\!\perp X_j | X_{-j}\}$$

Therefore:

Neighborhood selection for X_k is equivalent to feature selection where X_k represents Y and the remaining features $\{X_\ell : \ell \in \{1, 2, \dots, p\} \setminus \{k\}\}$ is feature set

And, to learn the full graph structure, run neighborhood selection for all p nodes $(X_1, X_2, \dots, X_p)$.

Neighborhood selection code (Lasso)

```
import numpy as np
from sklearn.linear_model import LassoCV
from sklearn.preprocessing import StandardScaler

def neighborhood_selection(X, alpha=0.5):
    n_samples, n_features = X.shape
    adjacency_matrix = np.zeros((n_features, n_features))

    # Standardize data
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    for j in range(n_features):
        # Response is X_j
        y = X_scaled[:, j]
        # Predictors are all other variables
        X_rest = np.delete(X_scaled, j, axis=1)

        # Fit Lasso with cross-validation if alpha is None
        if alpha is None:
            lasso = LassoCV(cv=5, positive=False).fit(X_rest, y)
            coef = lasso.coef_
        else:
            lasso = Lasso(alpha=alpha).fit(X_rest, y)
            coef = lasso.coef_

        # Fill adjacency matrix
        idx = 0
        for k in range(n_features):
            if k == j:
                continue
            if coef[idx] != 0:
                adjacency_matrix[j, k] = 1
            idx += 1

    # Symmetrize adjacency (OR rule)
    adjacency_matrix = np.maximum(adjacency_matrix, adjacency_matrix.T)
    return adjacency_matrix
```

Undirected graphical model summary

- Undirected graphical models represent relationships between nodes/features
- Concrete example: spatial data with natural non-directional dependence structure (e.g. image, geography, e.t.c.)
- Undirected graphical model learning can be represented as a neighborhood selection problem for each node
- Equivalent to solving p feature selection problems with $p - 1$ features
- Inherits all the same issues as feature selection
- Also need to resolve inconsistencies between neighborhood sets

Directed acyclic graphical (DAG) models

Directed graphical models:

$$G = (V, E)$$

where $V = (X_1, X_2, \dots, X_p)$, E directed edges

- *Acyclic* means there are no directed cycles/loops in the graph
- Directed acyclic graphical models (DAGs) all have a (possibly non-unique) causal ordering

$$(X_j, X_k) \in E \Leftrightarrow X_j \not\perp\!\!\!\perp X_k \mid \text{All features earlier in causal ordering}$$

Example 1

$$G = (V, E)$$

where $V = (X_1, X_2, X_3)$ and $E = \{(X_1, X_3), (X_2, X_3)\}$. Hence a causal ordering would be (X_1, X_2, X_3) (or X_2, X_1, X_3)

$$(X_1, X_2) \notin E \Leftrightarrow X_1 \perp X_2$$

$$(X_1, X_3) \in E \Leftrightarrow X_1 \not\perp X_3 | X_2$$

$$(X_2, X_3) \in E \Leftrightarrow X_2 \not\perp X_3 | X_1$$

Example 2

$$G = (V, E)$$

where $V = (X_1, X_2, X_3, X_4)$ and $E = \{(X_4, X_1), (X_1, X_3), (X_3, X_2), (X_1, X_2)\}$.
Hence a causal ordering is (X_4, X_1, X_3, X_2)

$$(X_1, X_2) \in E \Leftrightarrow X_1 \not\perp\!\!\!\perp X_2 | \{X_3, X_4\}$$

$$(X_1, X_3) \in E \Leftrightarrow X_1 \not\perp\!\!\!\perp X_3 | X_4$$

$$(X_4, X_1) \in E \Leftrightarrow X_1 \not\perp\!\!\!\perp X_4$$

$$(X_3, X_2) \in E \Leftrightarrow X_2 \not\perp\!\!\!\perp X_3 | \{X_1, X_4\}$$

$$(X_4, X_3) \notin E \Leftrightarrow X_3 \perp\!\!\!\perp X_4 | X_1$$

$$(X_4, X_2) \notin E \Leftrightarrow X_2 \perp\!\!\!\perp X_4 | \{X_1, X_3\}$$

Models/examples with intuitive directional/causal relationships

- **Sequential/time series data** (natural ordering) (e.g. Markov chains, state-space models, other ARIMA models)

$$(X_j, X_k) \in E \Leftrightarrow X_j \not\perp\!\!\!\perp X_k \mid \text{All features earlier in causal ordering}$$

intuitively means we should only condition on the past and not future nodes to determine the presence/absence of edges

- **Structural equation models** (may be tied to causality) (like burger and fries restaurant example)

$$(X_j, X_k) \in E \Leftrightarrow X_j \not\perp\!\!\!\perp X_k \mid \text{All features earlier in causal ordering}$$

intuitively means we should only condition on potential causes not effects to determine the presence/absence of edges

Time series model examples

General:

$$\mathbb{E}[X_t | X_1, X_2, \dots, X_{t-1}] = f_t(X_1, X_2, \dots, X_{t-1})$$

and observed nodes/time points $\{X_t\}_{t=1}^T$.

AR(k) process:

$$X_t = \sum_{j=1}^k a_j X_{t-j} + w_t$$

where $\{w_t\}_{t=k}^T$ independent Gaussian noise.

Each node has k parents corresponding to k most recent nodes (e.g. temperature, stock return e.t.c.)

What does DAG look like?

Structural equation model example

- Generative model widely used in social sciences/economics and other fields
- Example: Hamburger and fries restaurant (X_1, X_2, X_3) , where X_1 is hamburger quality, X_2 is fries quality, X_3 is whether restaurant survives more than 6 months

$$X_3 = \begin{cases} 1, & \text{if } X_1 + X_2 \geq 1 \\ 0, & \text{otherwise} \end{cases}$$

X_1, X_2 independent random variables/features

What does DAG look like?

DAG learning problem statement

Given observed feature data:

$$\{X_j^{(i)}\}_{j=1,2,\dots,p}^{i=1,2,\dots,n}$$

and pre-defined ordering (WLOG $X_1, X_2, \dots, X_p$) **Problem:** Can we learn/estimate the directed graphical model from observed data?

$$\mathcal{A} : \{X_j^{(i)}\}_{j=1,2,\dots,p}^{i=1,2,\dots,n} \Rightarrow \widehat{G} = (V, \widehat{E})$$

where $V = (X_1, X_2, \dots, X_p)$ and $\widehat{E}$ is set of directed edges where we can only have $(X_j, X_k) \in \widehat{E}$ if $j < k$

DAG learning and feature selection

For feature selection:

$$X_j \in \tilde{S} \Leftrightarrow Y \not\perp\!\!\!\perp X_j | X_{-j}$$

For DAG models:

$$(X_k, X_\ell) \in E \Leftrightarrow X_k \not\perp\!\!\!\perp X_\ell | X_S$$

where S is everything earlier in causal ordering.

Question: Is there a connection between whether there is a feature X_j in $\tilde{S}$ or there is an edge between $(X_k, X_\ell) \in E$?

Parent selection as feature selection

Define a *parent set* for node X_k (assume ordering is $X_1, X_2, \dots, X_p$):

$$\text{Pa}(X_k) = \{\ell : \ell \subset \{1, 2, \dots, k-1\}, (X_\ell, X_k) \in E\}$$

Recall that:

$$(X_\ell, X_k) \in E \Leftrightarrow X_k \not\perp\!\!\!\perp X_\ell | X_{1,2,\dots,\max(k-1,\ell-1)}$$

Therefore:

$$\text{Pa}(X_k) = \{\ell : \ell \subset \{1, 2, \dots, k-1\}, X_k \not\perp\!\!\!\perp X_\ell | X_S \text{ where } S = \{1, 2, \dots, k-1\}\}$$

For feature selection recall that:

$$\tilde{S} = \{j : j \subset \{1, 2, \dots, p\}, Y \not\perp\!\!\!\perp X_j | X_{-j}\}$$

Parent selection as feature selection cont.

- For feature selection we consider features $(X_1, X_2, \dots, X_p)$ that predict Y
- For parent selection in DAGs, for each $k = 1, 2, \dots, p$, we consider response X_k and features are $X_1, X_2, \dots, X_{k-1}$ (all nodes/features earlier in causal ordering)
- Once again, to learn full DAG, we need to solve p parent selection problems

Simulation example

- Consider 4-node example (X_1, X_2, X_3, X_4)

Structural equation model:

$$X_1 = \epsilon_1$$

$$X_2 = 0.8X_1 + \epsilon_2$$

$$X_3 = -0.6X_1 + \epsilon_3$$

$$X_4 = 0.7X_2 + 0.9X_3 + \epsilon_4$$

where ($\epsilon_1, \epsilon_2, \epsilon_3, \epsilon_4$) are independent random variables

Simulation code

```
: import numpy as np
import pandas as pd

np.random.seed(0)

n = 1000

# noise variables
e1 = np.random.normal(size=n)
e2 = np.random.normal(size=n)
e3 = np.random.normal(size=n)
e4 = np.random.normal(size=n)

# structural equations
X1 = e1
X2 = 0.8*X1 + e2
X3 = -0.6*X1 + e3
X4 = 0.7*X2 + 0.9*X3 + e4

df = pd.DataFrame({
    "X1":X1,
    "X2":X2,
    "X3":X3,
    "X4":X4
})

print(df.head())
```

	X1	X2	X3	X4
0	1.764052	1.967205	-2.591352	0.638100
1	0.400157	1.212600	-1.952064	-0.339316
2	0.978738	0.360676	-0.541108	-0.349011
3	2.240893	1.897429	-2.302910	-0.492789
4	1.867558	1.722100	-1.201346	-1.086598

Simulation DAG code

```
import statsmodels.api as sm

for target in ["X2", "X3", "X4"]:

    predictors = [v for v in ["X1", "X2", "X3"] if v != target]

    X = sm.add_constant(df[predictors])
    y = df[target]

    model = sm.OLS(y, X).fit()

    print("\nTarget:", target)
    print(model.pvalues)
```

```
Target: X2
const    6.933348e-01
X1       3.637091e-79
X3       9.538059e-01
dtype: float64
```

```
Target: X3
const    8.069484e-02
X1       2.578006e-53
X2       9.538059e-01
dtype: float64
```

```
Target: X4
const    5.734516e-01
X1       9.750382e-01
X2       3.179018e-85
X3       4.710213e-121
dtype: float64
```

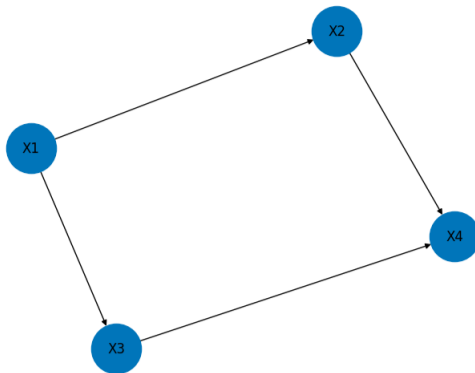
Simulation DAG visualization

```
G = nx.DiGraph()

edges = [
    ("X1", "X2"),
    ("X1", "X3"),
    ("X2", "X4"),
    ("X3", "X4")
]

G.add_edges_from(edges)

nx.draw(G, with_labels=True, node_size=2000)
plt.show()
```



Auto miles-per-gallon example

- Auto miles per gallon dataset (UCI Machine Learning repository)
- $n = 387$, $p = 6$
- X_1 : cylinders; X_2 : displacement; X_3 : horsepower; X_4 : weight; X_5 : acceleration; X_6 : miles per gallon
- Natural ordering $(X_1, X_2, \dots, X_6)$

Auto miles-per-gallon code

```
import pandas as pd
import statsmodels.api as sm
import pandas as pd

df = pd.read_csv("auto-mpg.csv")
df = df[['cylinders', 'displacement', 'horsepower', 'weight', 'acceleration', 'mpg']]
df = df.apply(pd.to_numeric, errors='coerce')
df = df.dropna()
```

```
import statsmodels.api as sm

ordering = [
    'cylinders',
    'displacement',
    'horsepower',
    'weight',
    'acceleration',
    'mpg'
]

edges = []

for j in range(1, len(ordering)):
    target = ordering[j]
    predictors = ordering[:j]

    X = sm.add_constant(df[predictors])
    y = df[target]

    model = sm.OLS(y, X).fit()

    for var, p in model.pvalues.drop("const").items():
        if p < 0.05:
            edges.append((var, target))

print(edges)

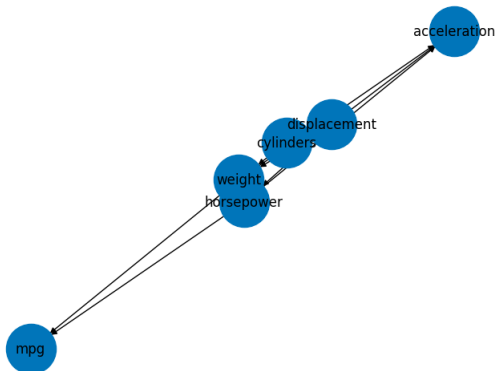
[('cylinders', 'displacement'), ('displacement', 'horsepower'), ('cylinders', 'weight'), ('displacement', 'weight'), ('horsepower', 'weight'), ('displacement', 'acceleration'), ('horsepower', 'acceleration'), ('weight', 'acceleration'), ('horsepower', 'mpg'), ('weight', 'mpg')]
```

Auto miles-per-gallon DAG code

```
import networkx as nx
import matplotlib.pyplot as plt

G = nx.DiGraph()
G.add_edges_from(edges)

nx.draw(G, with_labels=True, node_size=2000)
plt.show()
```



Carbon dioxide time series example

- Single attribute: monthly atmospheric carbon dioxide concentration (parts per million)
- Measured continuously since 1958
- Relate to Keeling curve
- As we observe gradually increasing over time

Carbon dioxide time series example

```
import statsmodels.api as sm
import pandas as pd

data = sm.datasets.co2.load_pandas().data
data = data.resample("ME").mean().interpolate()

y = data["co2"]
```

```
max_lag = 12

df = pd.DataFrame({"y":y})

for l in range(1,max_lag+1):
    df[f"lag{l}"] = y.shift(l)

df = df.dropna()
```

```
import statsmodels.api as sm

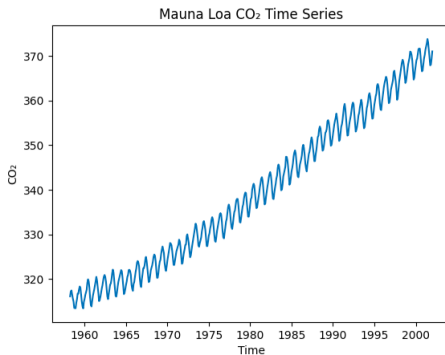
X = df[[f"lag{i}" for i in range(1,13)]]
X = sm.add_constant(X)

model = sm.OLS(df["y"],X).fit()

print(model.summary())
```

Carbon dioxide time series plot

```
plt.figure()
plt.plot(y)
plt.title("Mauna Loa CO2 Time Series")
plt.xlabel("Time")
plt.ylabel("CO2")
plt.show()
```



Carbon dioxide time series results

```

=====
                        OLS Regression Results
=====
Dep. Variable:          y      R-squared:          1.000
Model:                  OLS    Adj. R-squared:      0.999
Method:                  Least Squares    F-statistic:      8.457e+04
Date:                    Mon, 16 Mar 2026    Prob (F-statistic):    0.00
Time:                    02:23:15    Log-Likelihood:      -225.39
No. Observations:        514    AIC:                476.8
Df Residuals:            501    BIC:                531.9
Df Model:                12
Covariance Type:        nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	-1.6005	0.356	-4.495	0.000	-2.300	-0.901
lag1	0.9422	0.045	21.124	0.000	0.855	1.030
lag2	-0.2268	0.058	-3.889	0.000	-0.341	-0.112
lag3	-0.1348	0.059	-2.283	0.023	-0.251	-0.019
lag4	-0.0051	0.059	-0.086	0.932	-0.121	0.111
lag5	0.0718	0.059	1.217	0.224	-0.044	0.188
lag6	0.0666	0.059	1.131	0.259	-0.049	0.182
lag7	-0.0533	0.059	-0.909	0.364	-0.169	0.062
lag8	-0.0938	0.059	-1.597	0.111	-0.209	0.022
lag9	0.0462	0.059	0.784	0.433	-0.070	0.162
lag10	0.0172	0.059	0.294	0.769	-0.098	0.132
lag11	0.4238	0.058	7.330	0.000	0.310	0.537
lag12	-0.0480	0.045	-1.078	0.281	-0.135	0.039

```

=====
Omnibus:                2.354    Durbin-Watson:        2.022
Prob(Omnibus):          0.308    Jarque-Bera (JB):      2.331
Skew:                   0.064    Prob(JB):              0.312
Kurtosis:               3.304    Cond. No.              2.50e+04
=====

```

Carbon dioxide results overview

- Lags 1, 2, 3 and 11 are significant
- Lags 1, 2 and 3 reflect proximity
- Lag 11 is a seasonal effect (almost 12 months but not exactly)
- What does DAG look like?

Parent selection for DAGs vs neighborhood selection for undirected graph learning

DAGs

Parents

Condition on past based on ordering

Directional

Local and global equivalent

No inconsistent edges

Ordering required

UGs

Neighbors

Condition on all nodes/features

Non-directional

Local and global not equivalent

Edges possibly inconsistent

No ordering

DAG models naturally generative whereas UGs not

- With ordering, we can define, global distribution through local conditional distributions
- Reasons: chain rule
- Much more challenging (often impossible) for undirected graphical models

Chain rule:

$$f(x_1, x_2, \dots, x_p) = \prod_{j=1}^p f_j(x_j | x_{\text{Pa}(X_j)})$$

where $f_j(x_j | x_{\text{Pa}(X_j)})$ is conditional distribution for each x_j given its parents (like structural equation model)

Recall: Simulation example

- Consider 4-node example (X_1, X_2, X_3, X_4)

Structural equation model:

$$X_1 = \epsilon_1$$

$$X_2 = 0.8X_1 + \epsilon_2$$

$$X_3 = -0.6X_1 + \epsilon_3$$

$$X_4 = 0.7X_2 + 0.9X_3 + \epsilon_4$$

where $(\epsilon_1, \epsilon_2, \epsilon_3, \epsilon_4)$ are independent random variables

Recall: Simulation undirected graphical model

```
def ols_graphical_model(df, alpha=0.05, rule="and"):

    nodes = df.columns
    p = len(nodes)

    selected = {node:set() for node in nodes}

    # neighborhood selection
    for target in nodes:

        predictors = [v for v in nodes if v != target]

        X = sm.add_constant(df[predictors])
        y = df[target]

        model = sm.OLS(y,X).fit()

        for var,pval in model.pvalues.drop("const").items():
            if pval < alpha:
                selected[target].add(var)

    # symmetrize
    edges = set()

    for i in nodes:
        for j in nodes:
            if i >= j:
                continue

            if rule == "and":
                cond = j in selected[i] and i in selected[j]
            else: # OR rule
                cond = j in selected[i] or i in selected[j]

            if cond:
                edges.add((i,j))

    return edges

edges = ols_graphical_model(df, alpha=0.05, rule="and")
print(edges)

{'X1', 'X3'}, {'X2', 'X4'}, {'X3', 'X4'}, {'X2', 'X3'}, {'X1', 'X2'}
```

Why is there an extra edge in the recovered undirected graph?

Why is there an extra edge in the recovered undirected graph?

- Why does edge (X_2, X_3) exist for UG but not DAG?
- Condition on child of two parents (like burger and fries example)
- $X_2 \not\perp X_3|X_4$ therefore $X_2 \not\perp X_3|\{X_1, X_4\}$
- Since UGs condition on all other nodes, potentially create new edges for parents with a common child
- For DAGs, we don't condition on X_4 (since later in causal ordering) and $X_2 \perp X_3|X_1$ so no edge in DAG